

ProPlan Export Schema

Reading guide for Claude — for generating documentation from a ProPlan JSON export.

You are being given a JSON file exported from ProPlan, a single-file Gantt / phase-planner tool. Your task is to produce documentation from it — status reports, phase summaries, dependency overviews, milestone schedules, etc. Read this guide before parsing.

1. Top-level structure

```
{
  "start":      "YYYY-MM" or "YYYY-MM-DD",    // project start anchor
  "end":        "YYYY-MM" or "YYYY-MM-DD",    // project end (visible window only)
  "colWidth":   <number>,                    // pixels per month in the UI – IGNORE for docs
  "activePhase": "PRS" | "DP" | "DO" | "FP" | null,
  "lanes":      [ /* Work Packages – see §2 */ ]
}
```

- **start** is the anchor for ALL bar positions. Everything else is expressed as months-from-start.
- Ignore **colWidth** — it's a UI rendering hint.
- Date fields may be YYYY-MM (legacy) or YYYY-MM-DD. If YYYY-MM, treat day as 1.

2. Lanes (Work Packages)

```
{
  "id":      "<short id>",
  "code":    "WP 7 – Adhesive selection",    // canonical: "WP <n> – <name>"
  "desc":    "<optional description>",
  "phase":   "PRS" | "DP" | "DO" | "FP" | null,
  "bars":    [ /* tasks / deliverables / milestones – see §3 */ ]
}
```

A **lane = a Work Package (WP)**. The WP number is the integer in code (regex `/WP\s*(\d+)/i`). Use it for stable identification. Lanes have no time of their own — their start/end is the min/max of their bars.

3. Bars (tasks, milestones, etc.)

```
{
  "id":      "<short id>",
  "type":     "work" | "buffer" | "leadtime" | "review" | "milestone",
  "startIdx": <number>,                // MONTHS from start (fractional, 0.25 = 1 week)
  "span":     <integer>,                // duration in WEEKS (0 for milestones)
  "buffer":   <integer>,                // optional %: extra buffer added to span; default 0
  "label":    "<task name>",
  "dependsOn": [ "<barId>", ... ],
  "phase":    "PRS" | "DP" | "DO" | "FP" | null,    // starting phase
  "phaseEnd": "PRS" | "DP" | "DO" | "FP" | null,    // optional: bleed-to phase
  "extendsPhase": true | false        // optional: opt out of phase gating
}
```

Critical unit conventions

- **startIdx** is in **MONTHS** (fractional, multiples of 0.25 → 1-week granularity).
- **span** is in **WEEKS** (integer ≥ 0).
- $\text{WEEKS_PER_MONTH} = 4$. So a bar ends at $\text{startIdx} + (\text{span} \times (1 + \text{buffer}/100)) / 4$ months.

Effective duration

```
effSpanWeeks = span × (1 + buffer/100)           // 0 for milestone
effSpanMonths = effSpanWeeks / 4
endMonths     = startIdx + effSpanMonths
```

A bar with $\text{span}=12$, $\text{buffer}=30$ has effective duration $12 \times 1.30 = 15.6$ weeks ≈ 3.9 months.

Converting startIdx to a real date

```
projectStart = parse(state.start)
barStartDate  = projectStart + (startIdx months)
barEndDate   = projectStart + (endMonths)
```

A “month” here is calendar-month based — $\text{startIdx} = 1.0$ = exactly 1 calendar month after start; 0.25 = 1 week after start, treating a month as 4 weeks.

Bar types

Type	Meaning
work	Standard task / deliverable.
buffer	Schedule buffer / contingency (standalone).
leadtime	External lead time (tooling order, supplier wait).
review	Phase review or gated review meeting.
milestone	Point-in-time event. $\text{span}=0$. Has a date but no duration.

Buffer can ALSO be attached to a work bar via the **buffer percentage** field — that's a buffer at the tail of a work item, hatched in the UI. The **buffer type** is a standalone buffer bar.

4. Phases (SOP-18 model)

Order: PRS → DP → DO → FP

- **PRS** — Product Requirements Specification
- **DP** — Design Process
- **DO** — Design Output
- **FP** — Final Product

Each task is tagged with phase (its starting phase). phaseEnd is optional and indicates the task “bleeds” into a later phase — it starts in phase but its tail extends into phaseEnd. Only the LAST non-bleeding task in a phase gates the next phase. extendsPhase: true exempts a task from gating entirely (e.g. long-running tests that shouldn't block phase closure).

Computing phase ranges (for summaries)

For each phase code, scan all NON-milestone bars:

- `phase.start = min startIdx of bars where bar.phase === code`
- `phase.end = max endMonths of bars where gatePhase === code`
- `gatePhase = (bar.phaseEnd ranks later than bar.phase) ? bar.phaseEnd : bar.phase`
- Exclude bars with extendsPhase: true
- Exclude milestones — they're point markers, not range definers

Phase closure gates

A phase closure gate sits at `max(thisPhase.end, nextPhase.start)`. The final phase (FP) has a **Project close** gate at its own end. Surface these as project milestones in documentation even though they aren't in the JSON.

5. Dependencies

dependsOn is a flat array of predecessor bar IDs. The graph is a DAG (cycles are rejected by the tool).

A dependency is satisfied when `succ.startIdx ≥ pred.endMonths`. The tool flags as a violation when `pred.endMonths - succ.startIdx > 1 week (0.25 months)`. For documentation, you can:

- Build the dep graph for critical-path narrative.
- Report any violations you find (means the data is not auto-sequenced).
- Topologically sort to describe execution order.

6. What you should produce

Default deliverables (pick what matches the user's request):

- **Project summary** — Lanes/WPs with total effort, phase distribution, calendar start/end.
- **Phase summary** — For each phase: start date, end date, duration, list of deliverables, gate date.
- **Milestone register** — All type: "milestone" bars with their dates, grouped by WP.
- **Dependency narrative** — Topo-sorted task order; flag any deps whose successor starts before the predecessor's end.
- **Risk flags** — Bars with `buffer > 25%`, `extendsPhase: true`, or bleeding (phaseEnd set).
- **Gantt-style table** — WP | Task | Start (date) | End (date) | Duration (wks) | Phase | Deps.

7. Output conventions

- **Always convert** `startIdx` **and** `endMonths` **to real dates** using `state.start` as the anchor. Don't print month indices in user-facing output.
- Spans → weeks. Durations longer than 8 weeks may also be expressed as months for readability (12 wk ≈ 3 mo).
- Lane label = strip "WP <n> — " prefix if you want a short name, or keep full code for tables.
- Don't expose internal IDs in human-readable output. Use labels.
- Buffer: surface it visibly if it's a meaningful share of the task (≥10%). Phrase as "12 wk +30% buffer (15.6 wk effective)".

8. Pitfalls

- span is **WEEKS**, `startIdx` is **MONTHS**. Mixing units is the #1 source of off-by-N errors.
- A bar without phase is un-tagged — exclude it from phase summaries.
- Lanes can be empty. Don't crash; skip them.
- `dependsOn` can reference IDs that have been deleted (orphans). Filter to existing bar IDs.
- Milestones have `span: 0` but are real dates — include them in the milestone register, NOT in duration roll-ups.
- The export's `start/end` describe the visible window, NOT necessarily the data range. The true range is min/max across all bars' computed dates.